

听过知识点 稳定做对题

LMCC第一轮认证

高频考点串讲与高效备考策略



中国人民大学附属中学

谷多玉

01

知道考什么

知识大纲

明确高频专题、核心概念

02

知道怎么考

题型规律

客观题

03

学会怎么判断

解题方法

抓关键词、辨概念、看参
数、顺流程

04

知道怎么复习

备考策略

高频优先、理知识树、AI
辅助

时间 <https://lmcc.ccf.org.cn/>

- 青少年组

- 报名时间：2026年8月25日17:00截止
- 缴费时间：2026年8月26日17:00截止
- 第一轮认证：2026年9月5日（周六） 9:30- 11:30，认证2小时
- 第二轮认证：2026年10月17日（周六） 9:00- 12:00，认证3小时

- 成年组

- 报名时间：2026年9月5日17:00截止
- 缴费时间：2026年9月6日17:00截止
- 第一轮考试：2026年9月19日（周六） 9:30-11:30，时长2小时
- 第二轮考试：2026年10月18日（周日） 9:00-12:00，时长3小时

LMCC第一轮认证

- 30道**选择题**，100分：
 - 前20题（每题3分，共60分，占分60%+）
侧重AI基础知识考察，得分的关键区块
 - 后10题（每题4分，共40分）
程序填空类选择题，聚焦实操相关知识点
- 通过策略：拿稳前20题基础分（60分），再从后10题中选择有把握的，即可稳妥过线。

LMCC大纲考点全景图

一. 人工智能基础概念	1
二. 大模型基础概念	2
三. 模型架构	4
四. 预训练技术	6
五. 指令微调	10
六. 人类对齐	12
七. 解码与部署	13
八. 提示学习	15
九. 复杂推理	17
十. 智能体	18
十一. 模型评测	20
十二. 模型伦理与安全	23

大模型的生命周期



一条主线、六大专题



认

AI与大模型基础

AI、机器学习、深度学习、大模型

它是什么

懂

模型原理

Token、Embedding、Transformer、注意力、预测

它怎么工作的

学

训练与对齐

预训练、微调、对齐、数据

它怎么学会的

用

提示与推理

部署、Prompt、RAG、思维链

怎么让它回答得更好

做

Agent与工具调用

Agent、API、工具、 workflow

怎么让它从说到做

守

安全、伦理与评价

偏见、隐私、版权、安全、评价

怎么判断它好不好、该不该这样用？

六大高频考点

认

AI与大模型基础

AI、机器学习、深度学习、大模型

它是什么

懂

模型原理

Token、Embedding、Transformer、注意力、预测

它怎么工作的

学

训练与对齐

预训练、微调、对齐、数据

它怎么学会的

用

提示与推理

部署、Prompt、RAG、思维链

怎么让它回答得更好

做

Agent与工具调用

Agent、API、工具、 workflow

怎么让它从说到做

守

安全、伦理与评价

偏见、隐私、版权、安全、评价

怎么判断它好不好、该不该这样用？

专题1: AI与大模型基础

- 人工智能
- 机器学习
- 深度学习
- 生成式人工智能
- 大语言模型



机器学习 VS 深度学习

特征	机器学习	深度学习
特征工程	手动设计特征（特征提取+选择）	自动学习特征表示
数据规模	中小规模数据有效	数据量越大，优势越明显
计算资源	CPU即可	GPU/TPU集群
典型算法	决策树、SVM、随机森林、K-means	CNN、RNN、Transformer
可解释性	相对可解释	"黑盒"，难以解释

例题

以下关于机器学习和深度学习的说法，正确的是：

- A. 深度学习是机器学习的一个分支，二者是包含关系
- B. 机器学习是深度学习的一个分支，二者是包含关系
- C. 机器学习和深度学习是完全独立的两个领域
- D. 深度学习可以在没有任何数据的情况下进行训练

以下关于人工智能、机器学习、深度学习、生成式AI、大语言模型之间关系的说法，正确的是：

- A. 生成式AI和机器学习是完全不同的两个领域
- B. 大语言模型就是人工智能的全部
- C. 深度学习是机器学习的一个分支，大语言模型是生成式AI的重要形式
- D. 生成式AI只包含大语言模型一种形式

专题1： AI与大模型基础

- 监督学习 vs 非监督学习
 - 监督学习： 从有标签的训练数据中学习输入到输出的映射关系，然后用这个映射去预测新数据的标签
 - 有监督微调
 - 无监督学习： 从无标签的数据中发现内在结构、模式或分布规律，而无需预先给定输出目标

区分维度	监督学习	非监督学习
数据标签	有标签（有答案）	无标签（无答案）
学习目标	预测输出（回归/分类）	发现数据内在模式
常见任务	分类、回归、目标检测	聚类、降维、异常检测
典型算法	线性回归、逻辑回归、决策树、SVM、神经网络	K-Means、PCA、t-SNE、DBSCAN、自编码器
评价方式	用测试集计算准确率、MSE等	无直接评价，常通过可视化和下游任务
应用场景	垃圾邮件分类、房价预测、图像识别	客户分群、特征压缩、数据可视化
是否需要标注	需要（成本高）	不需要（成本低）

例题

关于监督学习和非监督学习，以下哪个说法是正确的？

- A. 监督学习不需要标注数据，可以直接从原始数据中学习
- B. 非监督学习的主要目标是在有标准答案的情况下预测新数据
- C. 监督学习和非监督学习的核心区别在于训练数据是否包含人工标注的目标标签
- D. 非监督学习通常比监督学习需要更多的人工标注成本

以下哪个任务最适合使用非监督学习解决？

- A. 根据房屋面积和地理位置预测房价
- B. 将电商网站的用户根据购物行为划分为不同的群体
- C. 判断一张X光片是否显示有肺炎
- D. 根据历史天气数据预测明天的气温

专题1: AI与大模型基础

- **精确率** (说对的比例) :

在所有被预测为"正例"的样本中, 实际真的是"正例"的比

$$Precision = \frac{TP}{TP + FP}$$

- **召回率** (找到的比例) :

在所有真的是"正例"的样本中, 被模型正确识别出来的比例

$$Recall = \frac{TP}{TP + FN}$$

- **准确率**

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- **F1分数**: 精确率和召回率的调和平均值, 同时兼顾两者

$$F1 = 2 \times \frac{precision \times recall}{precision + recall}$$

	预测为正例 (Positive)	预测为负例 (Negative)
真实为正例	TP (True Positive)	FN (False Negative)
真实为负例	FP (False Positive)	TN (True Negative)

例题

癌症筛查模型中，将一名健康人误判为癌症患者，这种错误在混淆矩阵中属于_____，对应的指标会直接使_____降低。

- A. FN; 召回率
- B. FP; 召回率
- C. FP; 精确率
- D. TN; 精确率

在构建一个用于识别网络攻击的入侵检测系统时，为了尽可能降低漏掉真正攻击的风险，你应当优先优化的指标是：

- A. 精确率
- B. 召回率
- C. F1分数
- D. 准确率

例题

某电商平台需要部署一个虚假评论检测系统，从用户评论中自动识别并标记出虚假评论（正例）。现有三个候选系统（A、B、C）在同一个测试集上进行了评估，测试集中共有200条评论，其中真实虚假评论有60条，真实正常评论有140条。

各系统的混淆矩阵如下：

系统	TP（正确识别虚假）	FP（正常误判为虚假）	FN（虚假漏判为正常）	TN（正常判断为正常）
A	48	14	12	126
B	54	28	6	112
C	42	7	18	133

已知平台的实际需求是：该平台每天有海量评论，人工审核资源有限。如果大量正常评论被误标记为虚假，会严重浪费审核人力，并影响正常用户的体验。因此，平台更看重被标记为虚假的评论中，有多大比例是真正的虚假评论。请问会选哪个系统？

c，精准率最高 $tp/(tp+fp)$

六大高频考点

认

AI与大模型基础

AI、机器学习、深度学习、大模型

它是什么

懂

模型原理

Token、Embedding、Transformer、注意力、预测

它怎么工作的

学

训练与对齐

预训练、微调、对齐、数据

它怎么学会的

用

提示与推理

部署、Prompt、RAG、思维链

怎么让它回答得更好

做

Agent与工具调用

Agent、API、工具、 workflow

怎么让它从说到做

守

安全、伦理与评价

偏见、隐私、版权、安全、评价

怎么判断它好不好、该不该这样用？

专题2：大模型为什么会说话

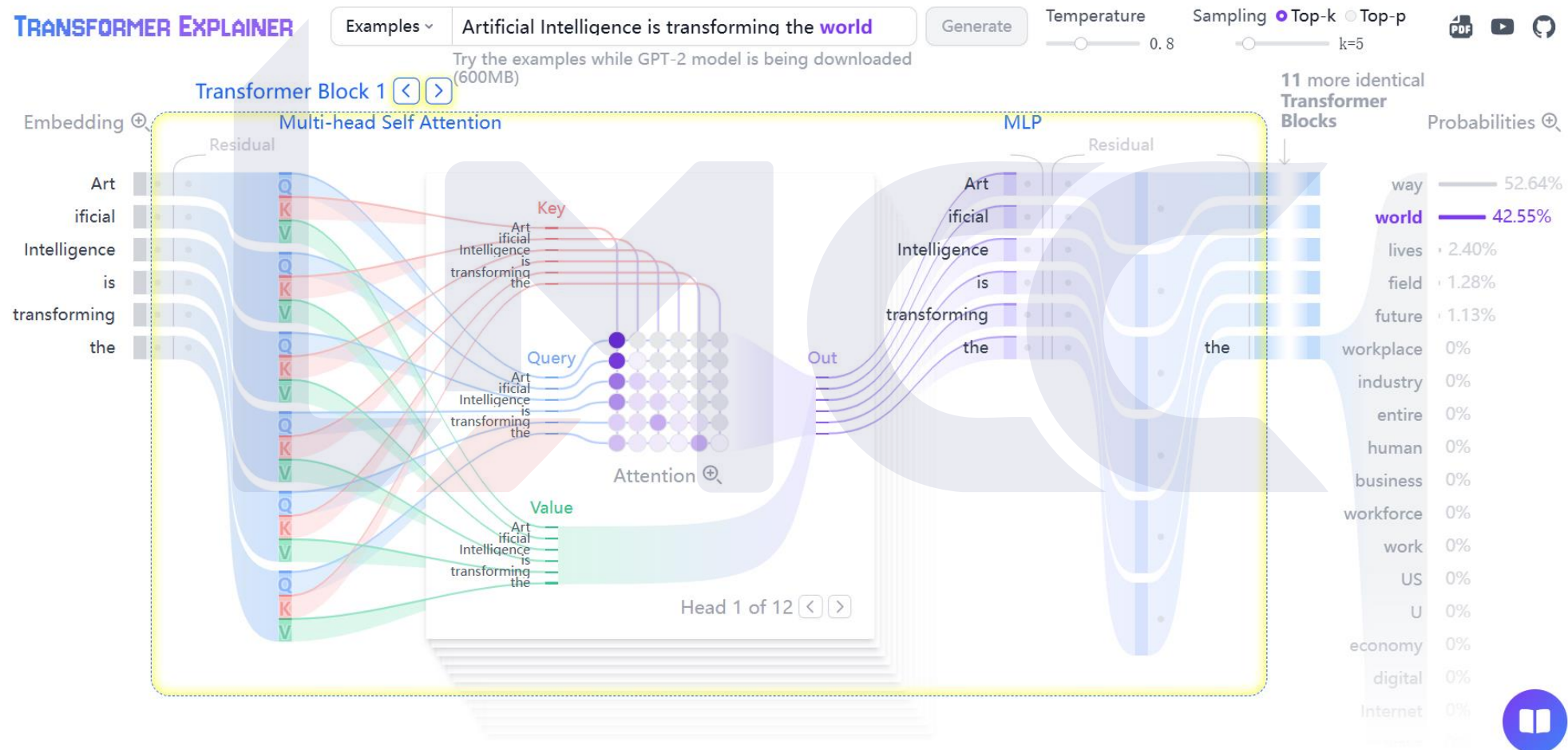
我今天来学校参加_____



专题2：大模型为什么会说话

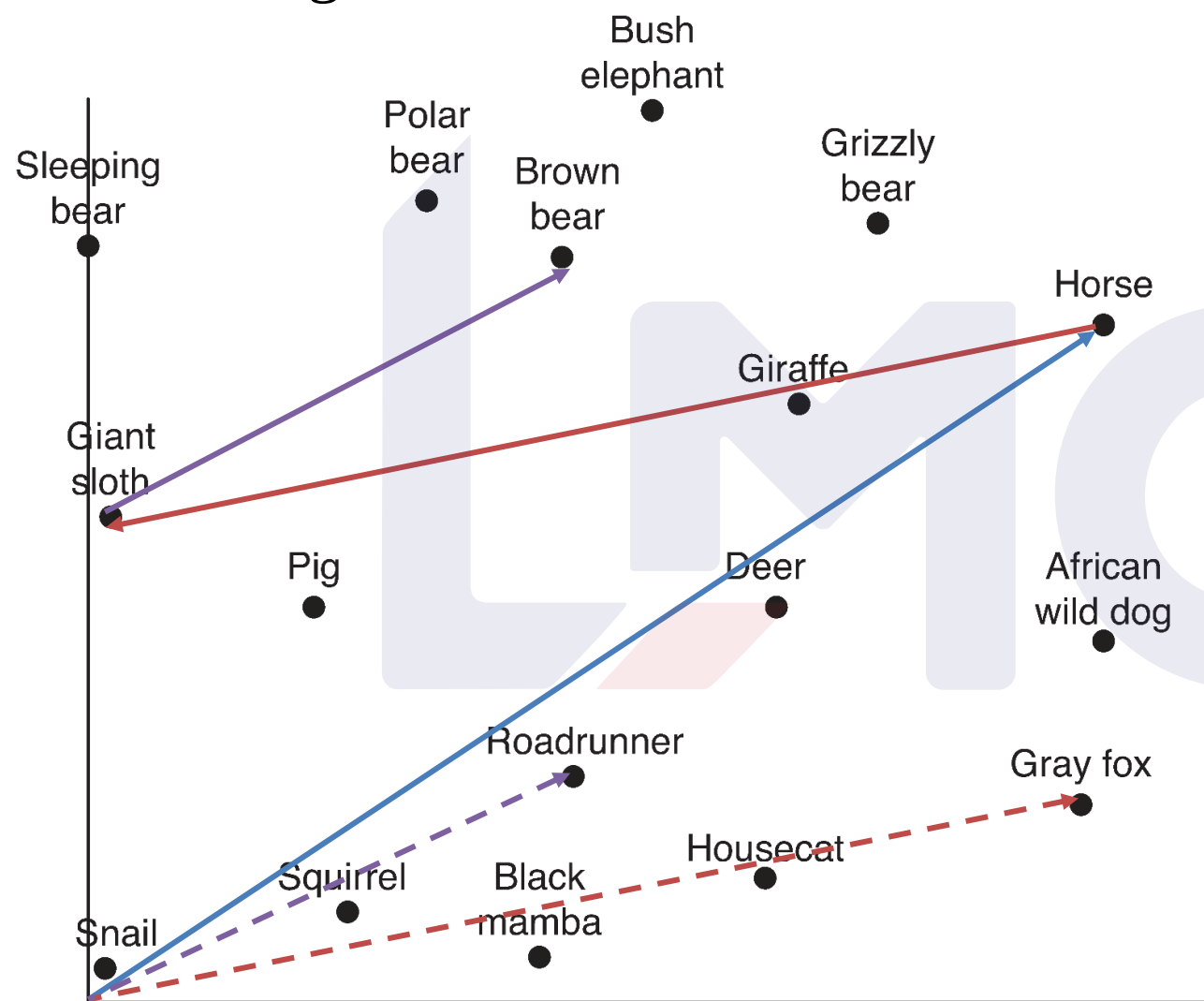
我今天来学校参加_____

大模型的工作：根据前面的内容，预测接下来最可能出现的Token。



专题2：大模型为什么会说话

- embedding



- 一匹马

- 像马，但不像狐狸

- 像马，不像狐狸，但像杜鹃一样快

- 国王-男+女=

专题2: 大模型为什么会说话

- Attention

Q: 我要查什么

K: 有什么可以被查

V: 真正的内容是什么

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

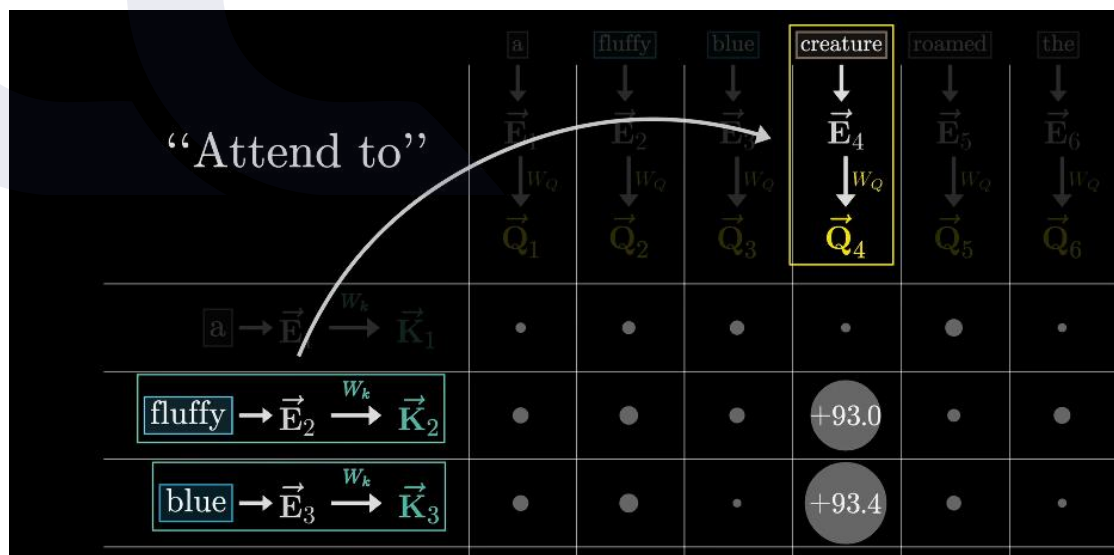
Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



3Blue1Brown ♂ LVE

【官方双语】直观解释注意力机制，Transformer的核心 | 【深度学习第6章】_哔哩哔哩_bilibili

https://www.bilibili.com/video/BV1TZ421j7Ke/?spm_id_from=333.1391.0.0&vd_source=d28761deb0ba37df33632b414ac8b6b4

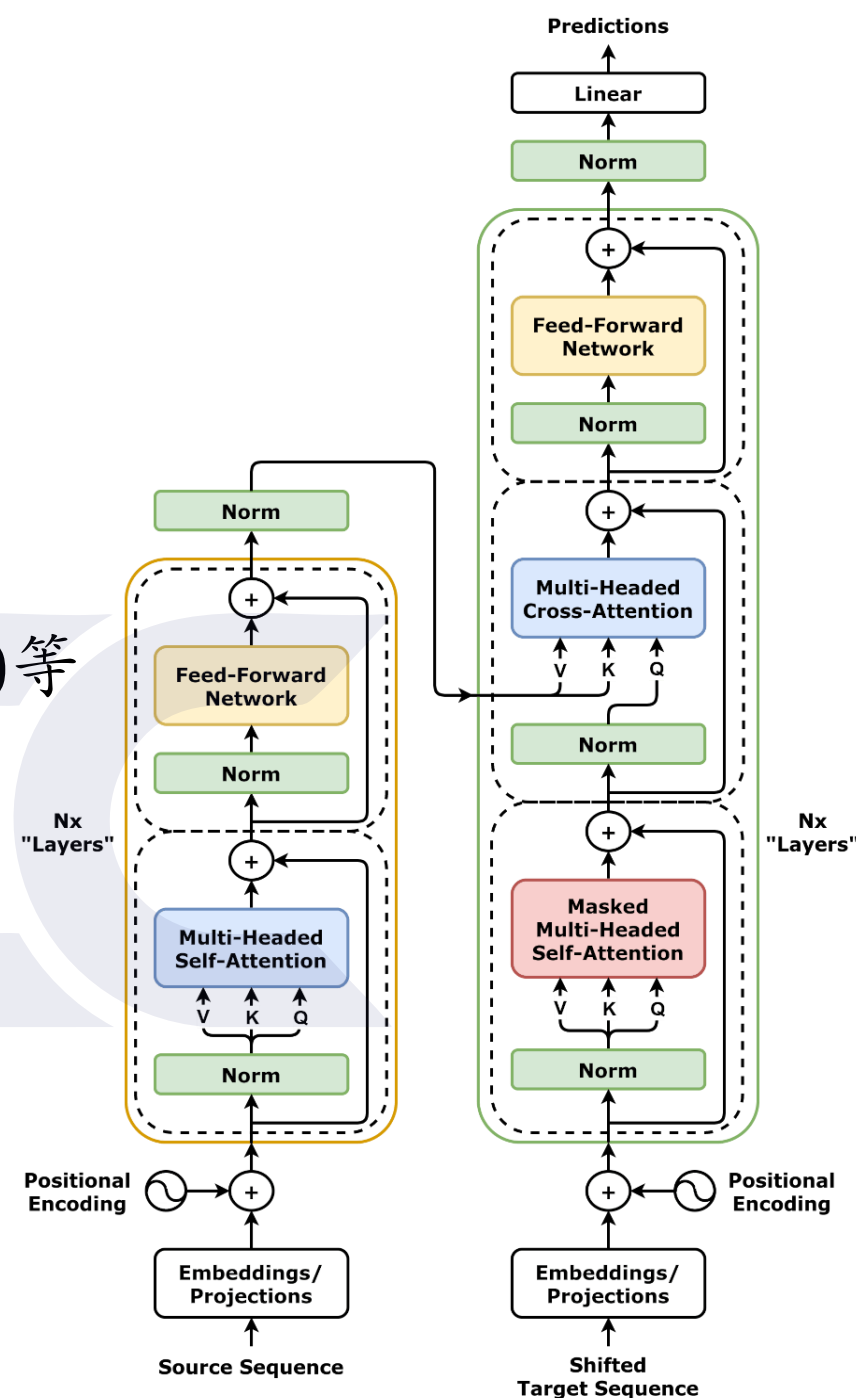
专题2：大模型为什么会说话

- 编码器：理解输入
- 解码器：生成输出

主流： Decoder-Only架构的大语言模型：

GPT、LLaMA、DeepSeek、Qwen3、kimi(K3)等

Encoder-Only架构：BERT



专题2：大模型为什么会说话

- Attention

Q: 我要查什么

K: 有什么可以被查

V: 真正的内容是什么

$$Q = X \cdot W_Q$$

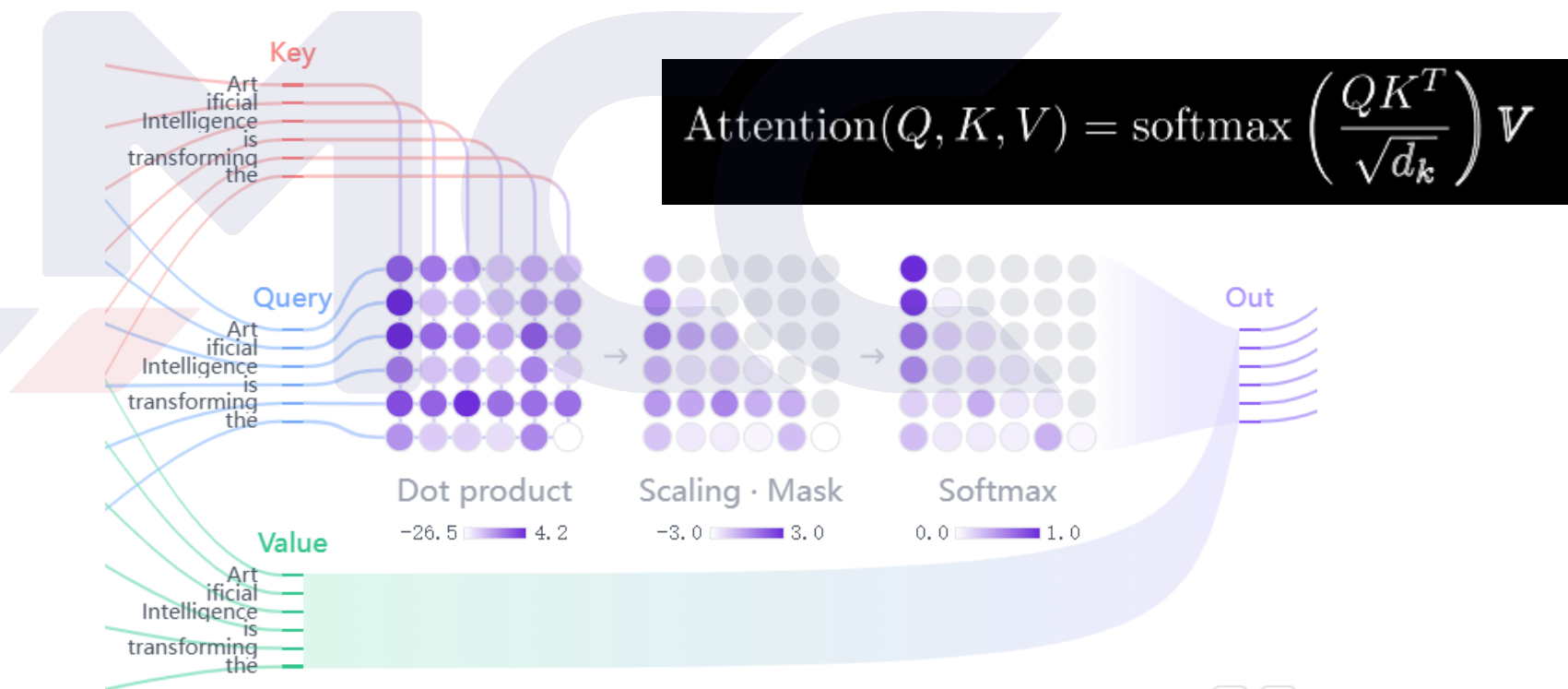
$$K = X \cdot W_K$$

$$V = X \cdot W_V$$

多头注意力

因果掩码：上三角

在Transformer的自注意力层中，Q、K、V都来自同一个输入序列，这就是“自”注意力的含义，它们由输入向量 X 分别经过三个不同的线性变换层投影得到



真题

Transformer架构中，使大语言模型能够有效处理长距离依赖关系的关键组件是？

- A. 前馈神经网络
- B. 层归一化
- C. 自注意力机制
- D. 残差连接

答案：C。自注意力机制让每个位置的词都能直接与所有位置的词建立联系，从而直接建模长距离依赖。

目前主流的生成式大语言模型（如GPT系列）的核心架构通常基于？

- A. 编码器模型
- B. 编码器-解码器模型
- C. 解码器模型
- D. 卷积神经网络模型

答案：C。GPT系列是典型的仅解码器（Decoder-only）模型，专为自回归文本生成任务设计

例题

在大模型处理文本的流程中，从原始文本到模型可计算的向量表示，其顺序是？

- A. 文本 → Token → 向量 (Embedding)
- B. 文本 → 向量 (Embedding) → Token
- C. Token → 文本 → 向量 (Embedding)
- D. 向量 (Embedding) → Token → 文本

关于位置编码，以下哪种说法是错误的？

- A. Transformer架构本身不具备感知输入顺序的能力。
- B. 位置编码的目的是为模型注入序列中Token的位置信息。
- C. 旋转位置编码（RoPE）是一种改进的位置编码方法，具有更好的外推性。
- D. 所有位置编码方法都是在输入嵌入向量上加上一个固定的、不可学习的向量。

A, 原始文本先被切分为Token（词元）序列，再通过嵌入层（Embedding Layer）将每个Token映射为稠密的数值向量。这是大模型“理解”文本的第一步。

D. 位置编码有不同类型，既有像原始Transformer中使用的固定正弦波编码，也有可学习的参数化位置编码。选项D的说法过于绝对。

例题：

以下关于自注意力机制的计算过程，描述正确的是？

- A. 直接计算输入向量之间的点积得到输出。
- B. 通过可学习的权重矩阵将输入向量投影为Q、K、V，计算Q与K的相似度，再用相似度加权V。
- C. 每个Token只与它前面的Token计算注意力，以保证自回归属性。
- D. 注意力权重使用Sigmoid函数进行归一化。

相较于传统的循环神经网络（RNN），Transformer架构最大的优势在于？

- A. 能够处理更短的文本。
- B. 必须按顺序处理序列，不能并行计算。
- C. 通过注意力机制可直接捕捉长距离依赖，且可并行处理序列。
- D. 参数量更小，训练更快。

B解析：这是自注意力机制的标准流程。选项C描述的是“掩码自注意力”（Masked Self-Attention），用于解码器。D选项错误，归一化应使用Softmax函数。

C解析：Transformer的核心优势在于其并行计算能力和直接建模长距离依赖的能力，这使其在大规模数据训练上远超RNN。

例题-归一化

Transformer中使用的是 Layer Normalization（层归一化），而不是 Batch Normalization。归一化的位置：每个子层之后（Post-Norm）或每个子层之前（Pre-Norm），当前主流是 Pre-Norm。代码中的对应：nn.LayerNorm(d_model)

```
import torch.nn as nn
class TransformerEncoderLayer(nn.Module):
    def __init__(self, d_model, nhead, dim_feedforward=2048, dropout=0.1):
        super().__init__()
        self.self_attn = nn.MultiheadAttention(d_model, nhead, dropout=dropout)
        self.linear1 = nn.Linear(d_model, dim_feedforward)
        self.linear2 = nn.Linear(dim_feedforward, d_model)
        self.dropout = nn.Dropout(dropout)
        self.norm1 = _____ # 请选择正确的归一化层
# 以下哪个是正确的归一化层？
# A. self.norm1 = nn.BatchNorm1d(d_model)
# B. self.norm1 = nn.LayerNorm(d_model)
# C. self.norm1 = nn.GroupNorm(d_model, d_model)
# D. self.norm1 = nn.InstanceNorm1d(d_model)
```

B

例题：归一化

```
class TransformerEncoderLayer(nn.Module):
    def __init__(self, d_model, nhead, dim_feedforward=2048, dropout=0.1):
        super().__init__()
        self.self_attn = nn.MultiheadAttention(d_model, nhead, dropout=dropout)
        self.linear1 = nn.Linear(d_model, dim_feedforward)
        self.linear2 = nn.Linear(dim_feedforward, d_model)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, src):
        # Pre-Norm 结构 (当前主流) 的正确顺序是:
        # A. src = self.self_attn(self.norm1(src), self.norm1(src), self.norm1(src))[0] + src ←
        # 如果是 Post-Norm, 顺序是 A (先计算再归一化)
        # B. src = self.norm1(src + self.self_attn(src, src, src)[0])

        attn_output = self.self_attn(_____, _____, _____)[0] # 填空
        src = src + self.dropout(attn_output)
        src = self.norm1(src)
        return src
```

正确答案 (Pre-Norm)

例题：激活函数

Transformer前馈网络（FFN）中使用的激活函数是 ReLU（原始Transformer），后续变体中使用 GELU（BERT、GPT-2/3、Llama等），SwiGLU（Llama、Qwen等主流模型使用），代码中的对应：F.relu、F.gelu、nn.SiLU()（等同于Swish/SwiGLU）

```
import torch.nn as nn
import torch.nn.functional as F
class FeedForward(nn.Module):
    def __init__(self, d_model, dim_feedforward=2048, dropout=0.1):
        super().__init__()
        self.linear1 = nn.Linear(d_model, dim_feedforward)
        self.linear2 = nn.Linear(dim_feedforward, d_model)
        self.dropout = nn.Dropout(dropout)
        self.activation = _____
    def forward(self, x):
        x = self.linear1(x)
        x = self.activation(x)
        x = self.dropout(x)
        x = self.linear2(x)
        return x
```

在原始Transformer中，前馈网络的激活函数是：

A. F.sigmoid

B. F.tanh

C. F.relu ← 正确答案（原始Transformer）

D. F.gelu

例题：注意力机制

多头注意力的核心公式： $Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$ ，缩放因子 $1/\sqrt{d_k}$ 的作用：防止点积过大，避免softmax梯度饱和。

num_heads：注意力头数，d_model 必须能被 num_heads 整除。

因果掩码（Causal Mask）用于Decoder-Only架构

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math
def scaled_dot_product_attention(query, key, value, mask=None):
    """
    query: [batch, heads, seq_len, d_k]
    key:   [batch, heads, seq_len, d_k]
    value: [batch, heads, seq_len, d_v]
    """
    d_k = query.size(-1)
    # 计算注意力分数（点积）
    scores = torch.matmul(query, key.transpose(-2, -1))

    scores = scores * _____
    if mask is not None:
        scores = scores.masked_fill(mask == 0, float('-inf'))

    attn_weights = F.softmax(scores, dim=-1)
    output = torch.matmul(attn_weights, value)
    return output, attn_weights
```

缩放因子应为：

- # A. 1.0
- # B. $1.0 / \text{math.sqrt}(d_model)$
- # C. $1.0 / \text{math.sqrt}(d_k)$
- # D. $1.0 / d_k$

C

例题：注意力机制

多头注意力的核心公式： $Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$ ，缩放因子 $1/\sqrt{d_k}$ 的作用：防止点积过大，避免softmax梯度饱和。

num_heads：注意力头数，d_model 必须能被 num_heads 整除。

因果掩码（Causal Mask）用于Decoder-Only架构

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.1):
        super().__init__()
        self.d_model = d_model
        self.num_heads = num_heads

        assert _____, "d_model must be divisible by num_heads"

        self.d_k = d_model // num_heads
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)
```

以下哪个条件是必须满足的？
A. $d_model > num_heads$
B. $d_model \% num_heads == 0$
C. $d_model == num_heads$
D. $d_model < num_heads * 2$

例题：注意力机制

多头注意力的核心公式： $Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$ ，缩放因子 $1/\sqrt{d_k}$ 的作用：防止点积过大，避免softmax梯度饱和。

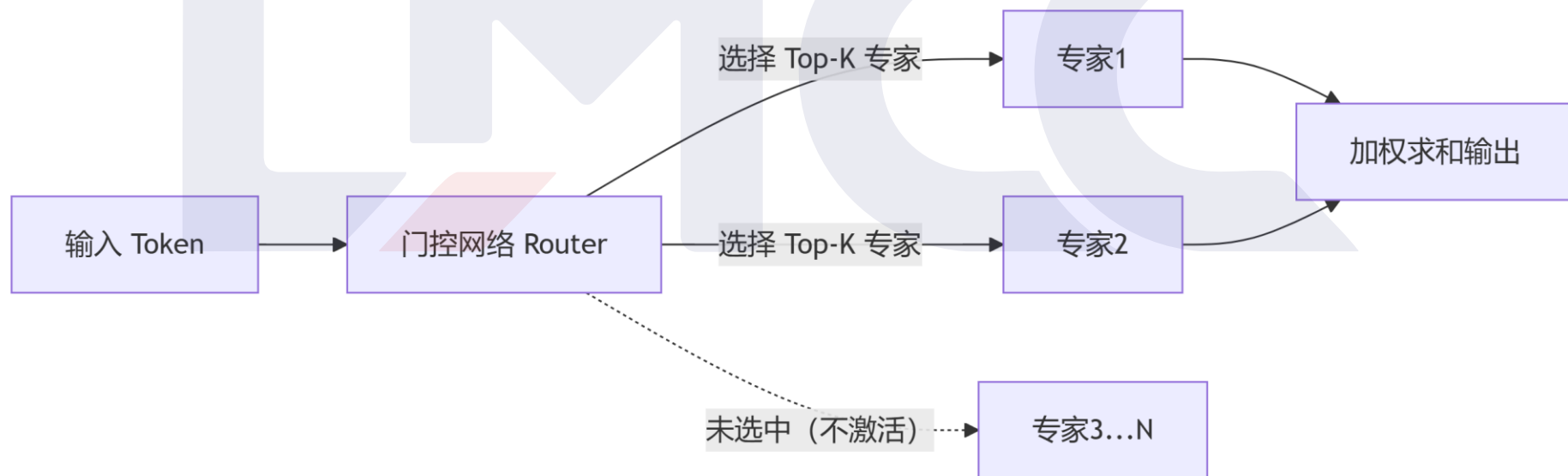
num_heads：注意力头数，d_model 必须能被 num_heads 整除。

因果掩码（Causal Mask）用于Decoder-Only架构

```
def create_causal_mask(seq_len, device):  
    """为自回归生成创建因果掩码（上三角掩码）"""  
    # 以下哪个操作能正确生成因果掩码？  
    # A. torch.triu(torch.ones(seq_len, seq_len, device=device), diagonal=0)  
    # B. torch.tril(torch.ones(seq_len, seq_len, device=device), diagonal=0) ← 正确答案  
    # C. torch.eye(seq_len, device=device)  
    # D. torch.ones(seq_len, seq_len, device=device)  
  
    mask = _____  
    return mask.unsqueeze(0).unsqueeze(1)
```

专题2：大模型为什么会说话

- **【3】 MoE (Mixture of Experts)** 混合专家模型，是一种通过动态路由机制，在每次计算时只激活模型中的一小部分参数的架构设计
- 理解成一个由众多专家组成的团队，每次处理任务时，只调动其中最擅长的那几位专家。这样做的好处是，模型的总参数量可以非常大，但实际推理时计算的参数量却小得多，大大节约了成本。



专题2：大模型为什么会说话

- MoE (Mixture of Experts) 混合专家模型
- 路由策略 (Routing Strategy)
 - 稀疏路由 (Sparse Routing)：每个token只激活 Top-K 个专家（通常 $K=1$ 或 2），其余专家不参与计算。这是MoE节省计算量的核心所在。
 - 负载均衡 (Load Balancing)：由于路由器可能总把token分配给某几个热门专家，导致其他专家“闲置”，MoE会在损失函数中加入辅助损失 (Auxiliary Loss)，鼓励token在各个专家间更均匀地分配。

例题：MoE

下列关于混合专家模型（MoE）的说法，正确的是：

- A. MoE层中的每个专家在每次前向传播时都会被激活
- B. MoE通过门控网络为每个token选择Top-K个专家，从而在保持大参数量的同时控制计算成本
- C. MoE的主要优势是减少了模型的总参数量
- D. MoE无需任何辅助损失函数即可正常工作

在MoE模型训练中，引入辅助损失（Auxiliary Loss）的主要目的是：

- A. 提高单个专家的准确率
- B. 鼓励token在不同专家之间均匀分配，防止部分专家闲置
- C. 减少模型的总参数量
- D. 加速路由器的收敛速度

专题2：大模型为什么会说话

编程例题：旋转位置编码 (RoPE) 的实现

旋转位置编码（Rotary Position Embedding, RoPE）是一种在Transformer模型中广泛应用的位置编码方法。它通过旋转矩阵对查询（Query）和键（Key）向量进行变换，以融入相对位置信息。RoPE 因其出色的外推能力和理论基础，被用于 LLaMA、Qwen、DeepSeek等主流大模型中。以下代码展示了 RoPE 的核心实现逻辑

```
import torch
import math
def precompute_freqs_cis(dim: int, seq_len: int, theta: float = 10000.0):
    """
    预计算旋转角度的复数表示  $(cis = \cos + i * \sin)$ 
    """
    # 计算频率 [dim/2]
    freqs = 1.0 / (theta ** (torch.arange(0, dim, 2)[: (dim // 2)].float() / dim))
    # 计算每个位置的旋转角度 [seq_len, dim/2]
    t = torch.arange(seq_len, device=freqs.device)
    freqs = torch.outer(t, freqs) # (seq_len, dim/2)
    # 转为复数形式
    freqs_cis = torch.polar(torch.ones_like(freqs), freqs) # (seq_len, dim/2)
    return freqs_cis
```

```
def apply_rotary_emb(xq: torch.Tensor, xk: torch.Tensor, freqs_cis: torch.Tensor):
```

```
    """
```

```
    将旋转位置编码应用到查询(Query)和键(Key)上。
```

```
    xq, xk: (batch, seq_len, n_heads, head_dim)
```

```
    freqs_cis: (seq_len, head_dim//2)
```

```
    """
```

```
    # 1. 将 xq 和 xk 的最后一个维度 (head_dim) 拆分为两半, 并转换为复数
```

```
    # 形状变换: (batch, seq_len, n_heads, head_dim) -> (batch, seq_len, n_heads, head_dim//2)
```

```
    xq_ = torch.view_as_complex(xq.float().reshape(*xq.shape[:-1], -1, 2))
```

```
    xk_ = torch.view_as_complex(xk.float().reshape(*xk.shape[:-1], -1, 2))
```

```
    # 2. 确保 freqs_cis 的维度与 xq_ 匹配 (batch, seq_len, n_heads, head_dim//2)
```

```
    freqs_cis = freqs_cis.unsqueeze(0).unsqueeze(2) # (1, seq_len, 1, head_dim//2)
```

```
    # 3. [1] 应用旋转: 将 xq_ 和 xk_ 与 freqs_cis 相乘
```

```
    xq_out = xq_ * freqs_cis
```

```
    xk_out = xk_ * freqs_cis
```

```
    # 4. 将复数结果转回实数, 并恢复原始形状
```

```
    xq_out = torch.view_as_real(xq_out).flatten(3)
```

```
    xk_out = torch.view_as_real(xk_out).flatten(3)
```

```
    return xq_out.type_as(xq), xk_out.type_as(xk)
```

```
def demo_rope():
```

```
    batch, seq_len, n_heads, head_dim = 2, 10, 8, 64
```

```
    # 随机生成 Query 和 Key
```

```
    xq = torch.randn(batch, seq_len, n_heads, head_dim)
```

```
    xk = torch.randn(batch, seq_len, n_heads, head_dim)
```

```
    # 预计算旋转角度
```

```
    freqs_cis = precompute_freqs_cis(head_dim, seq_len * 2)
```

```
    # 应用 RoPE
```

```
    xq_rope, xk_rope = apply_rotary_emb(xq, xk, freqs_cis)
```

```
    print(f"xq shape after RoPE: {xq_rope.shape}") # torch.Size([2, 10, 8, 64])
```

```
if __name__ == "__main__":
```

```
    demo_rope()
```

六大高频考点

认

AI与大模型基础

AI、机器学习、深度学习、大模型

它是什么

懂

模型原理

Token、Embedding、Transformer、注意力、预测

它怎么工作的

学

训练与对齐

预训练、微调、对齐、数据

它怎么学会的

用

提示与推理

部署、Prompt、RAG、思维链

怎么让它回答得更好

做

Agent与工具调用

Agent、API、工具、 workflow

怎么让它从说到做

守

安全、伦理与评价

偏见、隐私、版权、安全、评价

怎么判断它好不好、该不该这样用？

专题3：大模型怎么学会的

1

预训练 Pre-training

核心目标：获得基础能力

- 大数据 + 自监督学习 + 预测任务
- 学习语言、知识与通用规律
- 训练关键词：批次训练、学习率、优化器
- 典型任务：Next Token Prediction
- 结果：得到具备通用能力的基础模型

2

微调 Fine-tuning

核心目标：更适应目标任务

在已有预训练模型上，使用特定任务、领域或指令数据继续训练。

① 按训练目标分类

SFT 监督微调

Instruction
Tuning
指令微调

Domain Fine-
tuning
领域微调

② 按参数更新方式

Full Fine-tuning
全参数微调

PEFT
参数高效微调

LoRA / Adapter
Prompt Tuning
Prefix Tuning

关键辨析：Instruction Tuning 讲“教什么”；
LoRA 讲“参数怎么改”。

3

对齐 Alignment

核心目标：更符合人类期望

- 符合人类意图、偏好、价值与安全要求
- 目标：有用 (Helpful)
- 目标：无害 (Harmless)
- 目标：诚实 (Honest)
- RLHF：基于人类反馈的强化学习
- DPO：直接偏好优化

预训练：获得基础能力 → 微调：适应目标任务 → 对齐：行为更符合人类意图与偏好

预训练

步骤	内容	说明
数据收集	从互联网抓取海量文本	网页、书籍、论文、代码等，规模达数万亿Token
数据清洗	去重、过滤低质量内容	去除广告、乱码、重复内容，保留高质量文本
分词	将文本切分为Token	使用BPE等算法将文本转为数字ID序列
自监督学习	预测下一个词（Next Token Prediction）	模型根据前文预测下一个Token，计算损失并回传

预训练

- 自监督学习

预训练阶段不依赖人工标注数据，而是从数据本身构造监督信号

因果语言建模任务：给定前文，预测下一个词

以下关于预训练阶段“自监督学习”的说法，正确的是：

- A. 自监督学习需要大量人工标注的数据作为标签
- B. 自监督学习的监督信号来自数据本身，不需要额外的人工标注
- C. 自监督学习只适用于图像数据，不适用于文本数据
- D. 自监督学习无法用于大语言模型的训练

预训练

- 基于批次数据的训练方法

`per_device_train_batch_size` #单卡批次大小: 每张GPU在一次前向/反向传播中处理的样本数。

`gradient_accumulation_steps` # 梯度累积步数: 在更新参数前, 累积多少个mini-batch的梯度。

`global_batch_size` #全局批次大小: 模型每次参数更新时, 实际看到的样本总数。

`global_batch_size` =

`per_device_train_batch_size` × GPU数量 × `gradient_accumulation_steps`

批次大小	梯度稳定性	训练速度	泛化能力	显存占用
过大	梯度稳定, 收敛平滑	并行度高, 训练快	✗ 可能陷入锐利极小值, 泛化差	✗ 显存消耗极大
过小	✗ 梯度噪声大, 震荡明显	✗ 并行度低, 训练慢	可能收敛到平坦极小值, 泛化好	

预训练

- 基于批次数据的训练方法

在大模型训练中，显存是最稀缺的资源，因此通常采用以下策略：

策略一：小单卡批次 + 大梯度累积

```
training_args = TrainingArguments(  
    per_device_train_batch_size=1,    # 单卡每次只处理1个样本（显存友好）  
    gradient_accumulation_steps=32,    # 累积32次梯度  
    # 等效 global_batch_size = 1 × 8卡 × 32 = 256  
)
```

这种策略用时间换显存，让模型在显存不足时仍能模拟大批量训练的效果。

策略二：动态调整批次大小

训练初期：使用较小批次（梯度噪声有助于逃离不良局部最优）

训练后期：逐渐增大批次（梯度稳定有助于精细调优）

例题

当单卡显存不足时，为了达到较大的全局批次大小，最有效的方法是：

- A. 增大per_device_train_batch_size
- B. 增大gradient_accumulation_steps
- C. 增大学习率
- D. 减少gradient_accumulation_steps

在分布式训练中，per_device_train_batch_size=4，gradient_accumulation_steps=8，使用4张GPU，则global_batch_size为：

- A. 32
- B. 64
- C. 128
- D. 256

预训练

- 学习率：决定模型参数在每次更新时调整的步长大小

学习率大小	效果	问题
太大	参数更新步长过大	跨越最优解，Loss剧烈震荡，甚至梯度爆炸导致训练崩溃
太小	参数更新步长过小	训练极慢，可能陷入局部最优而无法逃脱

- 学习率预热

训练初期，模型参数是随机初始化的，梯度方向不稳定，幅度也可能很大。如果一开始就使用较大的学习率，容易导致梯度爆炸或训练不稳定。训练开始时，学习率从一个小值逐步增大到设定的最大学习率，待模型稳定后再按计划衰减。

预训练

- 优化器：优化器负责根据梯度更新模型的参数，核心问题是找到最快、最稳的下山路径

优化器	核心特点	适用场景	大模型场景
SGD	基础梯度下降，收敛慢	小规模、简单任务	几乎不用
Adam	自适应学习率，收敛快	大部分深度学习任务	早期常用
AdamW	Adam + 权重衰减解耦	大模型训练主流	最常用
AdaFactor	内存占用极低	受限环境下的超大模型	常用

```
# AdamW配置示例
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=3e-4,                # 学习率
    betas=(0.9, 0.95),     # 一阶/二阶动量衰减系数
    eps=1e-8,              # 防除零
    weight_decay=0.1       # 权重衰减
)
```

例题

在大型语言模型训练中采用学习率预热（Warm-up）策略，其主要原因是：

- A. 增加训练时间，提高模型精度
- B. 训练初期梯度不稳定，使用较小学习率有助于稳定训练
- C. 减少显存占用
- D. 加快推理速度

在AdamW优化器中，weight_decay参数的主要作用是：

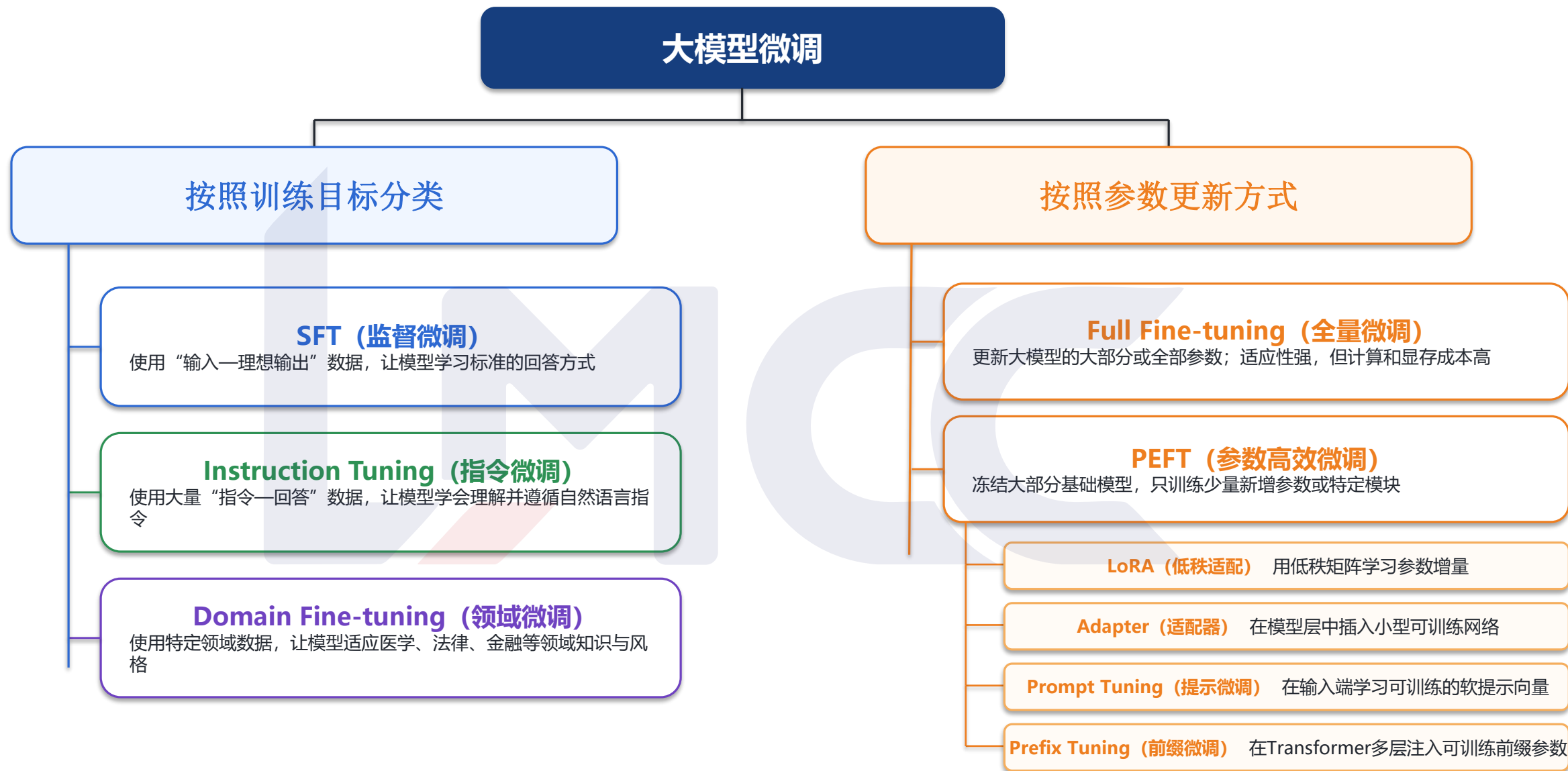
- A. 增大模型参数量
- B. 对大的参数施加额外惩罚，防止过拟合
- C. 增加训练速度
- D. 减少训练数据的规模

```

def pretrain():
    # ----- 超参数 -----
    vocab_size = 50257      # 类似 GPT-2 的词表大小
    d_model = 768          # 模型宽度
    num_heads = 12         # 注意力头数 (768 / 12 = 64)
    num_layers = 12        # 层数
    max_seq_len = 256      # 训练的序列长度
    batch_size = 8         # 单卡批次大小
    total_steps = 1000     # 演示用, 实际预训练可达数十万步
    # ----- 数据 -----
    dataset = DummyDataset(vocab_size, max_seq_len, batch_size)
    # ----- 模型 -----
    model = MiniGPT(vocab_size, d_model, num_heads, num_layers, max_seq_len)
    # ----- 优化器 -----
    optimizer = AdamW(model.parameters(), lr=3e-4, betas=(0.9, 0.95), weight_decay=0.1)
    # ----- 学习率调度器 -----
    scheduler = get_cosine_schedule_with_warmup(optimizer, num_warmup_steps=100, num_training_steps=total_steps)
    # ----- 训练循环 -----
    model.train()
    for step in range(total_steps):
        # 1. 获取一个批次
        x = dataset.get_batch()      # [batch, seq_len]
        # 2. 前向传播
        logits = model(x)            # [batch, seq_len, vocab]
        # 3. 计算损失: 预测下一个词
        shift_logits = logits[:, :-1, :].contiguous()
        shift_labels = x[:, 1:].contiguous()
        loss = F.cross_entropy(shift_logits.view(-1, vocab_size), shift_labels.view(-1))
        # 4. 反向传播
        optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0) # 梯度裁剪
        optimizer.step()
        scheduler.step()
        # 5. 监控
        if step % 100 == 0:
            current_lr = scheduler.get_last_lr()[0]
            print(f"Step {step:4d} | Loss: {loss.item():.4f} | LR: {current_lr:.2e}")

```

微调 在已有预训练模型上，使用特定任务、领域或指令数据继续训练。



大模型微调 Fine-Tuning

- 监督微调 SFT (Supervised Fine-Tuning)

用高质量的输入-输出对直接训练模型，让模型学会特定任务的映射关系
最直观、最常用的微调方式

目标是让模型掌握特定任务的输出格式和内容

通常后续是RLHF/偏好对齐的前置步骤

典型场景：对话模型初始化、代码生成、摘要生成

大模型微调 Fine-Tuning

- 指令微调 Instruction Tuning

让模型学会理解并遵循人类指令，学会如何按照要求做事，而不只是模仿训练数据中的固定模式。

与SFT的关键区别

	SFT	Instruction Tuning
数据多样性	单一任务	多任务、多领域混合
输入格式	原始文本/简单 prompt	结构化指令（“请总结”“请判断”.....）
目标	任务精度	泛化到未见过的指令

大模型微调 Fine-Tuning

- 领域微调 Domain Fine-Tuning

在特定专业领域数据（医疗、法律、金融、代码、科学等）上，对通用大模型进行进一步训练，使其：

掌握领域术语和知识（如医学中的"房颤消融术"、法律中的"要约邀请"）

理解领域特有的推理逻辑（如法律中的三段论、金融中的风险分析框架）

适配领域的表达风格（如病历的客观描述、法律文书的严谨措辞）

大模型微调 Fine-Tuning

- 全参数微调 (Full Fine-Tuning)

更新模型的所有参数，是最传统、最直接的微调方式。

优点：效果最好，模型能够充分适应新任务

缺点：需要大量显存、训练成本高、容易过拟合

适用场景：7B以下的模型、有充足算力、任务与预训练差异较大

大模型微调 Fine-Tuning

- 参数高效微调 PEFT (Parameter-Efficient Fine-Tuning)

冻结大部分预训练参数，仅更新少量额外参数

基于添加 (Additive)

→ 引入新参数模块

Adapter适配器

插入小型适配器模块

Prompt Tuning

学习输入端 Soft Prompt

Prefix Tuning

在多层注入可训练 Prefix

IA³

学习少量缩放向量

基于选择 (Selective)

→ 选择部分原参数更新

BitFit

只微调 Bias 偏置参数

Layer-wise Fine-tuning

选择部分网络层进行更新

基于重参数化 (Reparameterization)

→ 用更高效的参数形式表示更新

LoRA

低秩矩阵学习参数增量

DoRA

分解权重幅值与方向进行适配

PiSSA

基于奇异值结构进行低秩初始化/适配

MoRA

调整低秩适配的表示容量

混合 (Hybrid)

→ 组合多种 PEFT 思路

组合示例

LoRA + Prompt Tuning

大模型微调 Fine-Tuning

- 参数高效微调 PEFT (Parameter-Efficient Fine-Tuning)

Prompt Tuning 提示微调

在输入端加入一串可训练的“软提示 (Soft Prompt)”或“虚拟Token”。这些Token不是人能直接读懂的自然语言，而是一组可以通过训练自动学习的向量。

例如原始输入是：“请解释什么是人工智能”

Prompt Tuning 会变成：[P1][P2][P3][P4] + “请解释什么是人工智能”

大模型微调 Fine-Tuning

- 参数高效微调 PEFT (Parameter-Efficient Fine-Tuning)

Prefix Tuning 前缀微调

训练一组“软提示”，但不是只加在最开始的输入Embedding处，而是把一组可训练的“前缀参数”注入到 Transformer 的多个层中，尤其常见于注意力模块相关位置

Prompt Tuning:

Soft Prompt



输入Embedding



Transformer第1层



Transformer第2层



Transformer第3层

Prefix Tuning

输入



Transformer第1层 ← Prefix



Transformer第2层 ← Prefix



Transformer第3层 ← Prefix



输出

大模型微调 Fine-Tuning

- 参数高效微调 PEFT (Parameter-Efficient Fine-Tuning)

LoRA 低秩适配 Low-Rank Adaptation

当前大模型微调中最主流的参数高效微调 (PEFT) 方法。它通过引入低秩矩阵来模拟参数更新, 在保持模型性能的同时, 大幅降低显存和计算需求。微调过程中, 模型权重的更新 ΔW 虽然维度很高, 但其本质是低秩的——即可以用两个小矩阵的乘积来近似表示

$$h = W_0 \cdot x + \Delta W \cdot x$$

$$h = W_0 \cdot x + B \cdot A \cdot x$$

A: 形状 $r \times d$ (r 远小于 d)

B: 形状 $d \times r$ (r 远小于 d)

训练时只更新 A 和 B, 不更新 W_0

r 是 LoRA 最重要的超参数

实际前向传播

```
output = W_0 @ x + (lora_alpha / r) * (B @ A @ x)
```

lora_alpha 是缩放因子, 控制低秩矩阵的影响力

alpha / r 是实际缩放比例

常见配置: $r=8, \text{alpha}=16 \rightarrow$ 缩放比例为 2

Lora低秩适配-样题

低秩适配方法 LoRA (Low-Rank Adaptation) 是一种参数高效微调方法，通过只训练部分低秩矩阵插入至原始模型的关键层，实现大语言模型的高效定制。与传统微调相比，LoRA 显著减少了训练参数量和显存需求，适用于资源受限场景下的大模型下游训练。一个典型训练场景如下：

- 模型：**Qwen-2.5-7B-Instruct**；LoRA 仅插入注意力机制中关键的两个投影层。
- 资源：**单卡 16GB 显存**；输入序列长度上限 **2048**。
- 训练策略：需将全局 batch size 控制在 8 左右，在显存足够的情况下**尽量快**的进行训练。
- 精度：启用半精度训练以降低显存占用。
- 产出：训练完成后，仅保存 LoRA adapter 权重，供下游推理使用。

```
Python
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM,
TrainingArguments, Trainer
from peft import LoraConfig, get_peft_model
```


Lora低秩适配-样题

```
model_name = "Qwen/Qwen2.5-7B-Instruct"

# 1) 加载分词器
tokenizer = [21]
model = AutoModelForCausalLM.from_pretrained(
    model_name, device_map="auto", torch_dtype=torch.float16,
    trust_remote_code=True
)

# 2) 配置 LoRA
lora_config = LoraConfig(
    [22],
    target_modules=[23],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)

model = get_peft_model(model, lora_config)

# 3) 定义 Trainer (假设 train_dataset 已经准备好)
training_args = TrainingArguments(
    output_dir="./lora-qwen2.5-7b",
    [24],
    num_train_epochs=3,
    learning_rate=2e-4,
    [25],
    logging_steps=10,
    save_strategy="epoch"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    tokenizer=tokenizer,
)
```

```
trainer.train()
```

4) 保存 LoRA 权重

```
model.save_pretrained("./lora-qwen2.5-7b-lora")
```

21. Qwen2.5 可能包含自定义分词器/处理逻辑, 要求与模型端一致的 remote code。请选择一种能正确加载该模型分词器的方式:

- A. AutoTokenizer.from_pretrained(model_name, return_tensors="pt")
- B. AutoTokenizer.load(model_name)
- C. AutoTokenizer(model_name)
- D. AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)

22. 请参考题目描述的计算资源, 在尽量保持训练稳定和速度的前提下, 选择常见且相对节省显存的 LoRA 配置:

- A. r=64, lora_alpha=32,
- B. r=8, lora_alpha=64,
- C. r=8, lora_alpha=16,
- D. r=32, lora_alpha=8,

23. 为了配置 LoRA 权重的注入位置, 以下代码更合理的是:

- A. ["mlp", "norm"]
- B. "all_linear_layers"
- C. ["k_proj", "o_proj"]
- D. ["q_proj", "v_proj"]

Lora低秩适配-样题

```
model_name = "Qwen/Qwen2.5-7B-Instruct"

# 1) 加载分词器
tokenizer = [21]
model = AutoModelForCausalLM.from_pretrained(
    model_name, device_map="auto", torch_dtype=torch.float16,
    trust_remote_code=True
)

# 2) 配置 LoRA
lora_config = LoraConfig(
    [22],
    target_modules=[23],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)

model = get_peft_model(model, lora_config)

# 3) 定义 Trainer (假设 train_dataset 已经准备好)
training_args = TrainingArguments(
    output_dir="./lora-qwen2.5-7b",
    [24],
    num_train_epochs=3,
    learning_rate=2e-4,
    [25],
    logging_steps=10,
    save_strategy="epoch"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    tokenizer=tokenizer,
)
```

```
trainer.train()
```

4) 保存 LoRA 权重

```
model.save_pretrained("./lora-qwen2.5-7b-lora")
```

24. 显存限制和内存溢出 (Out Of Memory, OOM) 风险下, 以下配置最稳妥的是:

- A. per_device_train_batch_size=2, gradient_accumulation_steps=4,
- B. global_train_batch_size=8, gradient_accumulation_steps=4,
- C. global_train_batch_size=8, per_device_train_batch_size=1,
- D. per_device_train_batch_size=1, gradient_accumulation_steps=8,

25. 为了配置训练的数据类型, 以下代码更合理的是:

- A. fp32=True,
- B. fp16=True,
- C. training_type="fp32",
- D. training_type="bf16",

例题

LoRA (Low-Rank Adaptation) 的核心思想是:

- A. 降低模型的层数, 从而减少参数量
- B. 将权重的更新量近似为两个低秩矩阵的乘积, 只训练这两个小矩阵
- C. 将模型的权重全部替换为随机初始化的低秩矩阵
- D. 在训练过程中逐渐降低模型参数的精度

在使用LoRA微调LLaMA模型时, `target_modules=["q_proj", "v_proj"]` 的含义是:

- A. 在所有注意力层上应用LoRA
- B. 只在模型的输出层应用LoRA
- C. 只在词嵌入层应用LoRA
- D. 在所有全连接层应用LoRA

例题

以下关于大模型微调阶段学习率设置的说法，正确的是：

- A. 微调阶段应使用比预训练阶段更大的学习率
- B. LoRA微调通常可以使用比全参数微调稍高的学习率
- C. 全参数微调的学习率通常设置为 $1e-2$
- D. 所有微调场景都应使用相同的学习率

某团队想在单卡16GB显存下对7B模型进行LoRA微调，以下配置中_____最合理？

- A. `per_device_train_batch_size=4`，梯度累积=1
- B. `per_device_train_batch_size=1`，梯度累积=8
- C. `per_device_train_batch_size=8`，梯度累积=2
- D. `per_device_train_batch_size=2`，梯度累积=16

对齐

- 让大语言模型的行为和输出与人类的价值观、意图和社会规范保持一致
- 经过预训练和SFT后，模型已经具备了强大的语言能力和对话能力，但仍存在三个核心问题：有害内容、幻觉、不配合。对齐的目标是让模型同时满足：有用、无害、诚实HHH

3

对齐 Alignment

核心目标：更符合人类期望

- 符合人类意图、偏好、价值与安全要求
- 目标：有用 (Helpful)
- 目标：无害 (Harmless)
- 目标：诚实 (Honest)
- RLHF：基于人类反馈的强化学习
- DPO：直接偏好优化

RLHF 基于人类反馈的强化学习

阶段 1 | 监督微调 SFT

高质量“问题—理想回答”
→ 微调预训练模型

Prompt + 人类示范回答



得到基线模型 π_{SFT}

作用：先让模型学会基本的指令遵循与高质量回答方式。

阶段 2 | 奖励建模 RM

RLHF的关键：学习“人类更喜欢哪个回答”

同一 Prompt → 回答 A / B / C
人类排序：A > B > C



训练独立 Reward Model
输入：问题 + 回答 → 输出：标量分数

本质：RM成为“人类偏好的代理评分员”，自动预测一个回答有多符合偏好。

阶段 3 | PPO 强化学习

π_{RL} 生成回答 → RM评分 → PPO更新

① π_{RL} 根据 Prompt 生成回答

② Reward Model 给出奖励 r_θ

③ 与冻结参考模型 π_{ref} 计算 KL 惩罚

④ 最终奖励： $r_\theta - \beta \cdot \text{KL}(\pi_{\text{RL}} \parallel \pi_{\text{ref}})$

⑤ PPO 根据奖励更新 π_{RL}

KL约束：防止模型为了“讨好RM”偏离原模型太远。

优势

对复杂偏好优化灵活，奖励模型可迭代更新，效果潜力高。

挑战

流程复杂、计算成本高；PPO工程实现与训练稳定性要求较高。

SFT先示范 → RM学会评分 → PPO依据“奖励 - KL惩罚”优化主模型

DPO: 直接偏好优化 (Direct Preference Optimization)

核心原理 | 把“偏好排序”直接变成策略优化目标

偏好数据: 同一 Prompt 下, 获胜回答 y_{win} > 失败回答 y_{lose}

① RLHF目标 最大化奖励, 同时用 KL 散度约束策略不要偏离参考模型

② 得到最优策略关系 $\pi^*(y|x) \propto \pi_{ref}(y|x) \cdot \exp(r(y|x)/\beta)$

③ 反解奖励函数 把奖励 r 表示成策略 π 与参考策略 π_{ref} 的函数

④ 代入偏好模型 代入 Bradley-Terry: $P(y_1 > y_2) = \sigma(r(y_1) - r(y_2))$

⑤ 消去显式奖励函数 最终损失只依赖 π 、 π_{ref} 与偏好回答对

$$L_{DPO} = -E[\log \sigma(\beta \cdot (\log \pi(y_{win}|x)/\pi_{ref}(y_{win}|x) - \log \pi(y_{lose}|x)/\pi_{ref}(y_{lose}|x)))]$$

直观理解: 相对参考模型, 提高“获胜回答”的偏好优势, 压低“失败回答”的偏好优势。

DPO典型流程 | 仅两步

① SFT: 先得到基线 / 参考模型



② 偏好对 → DPO Loss → 更新策略模型

不需要单独训练 Reward Model, 也不需要运行 PPO

与经典RLHF最核心的区别

RLHF: 偏好 → RM → PPO → 模型

DPO: 偏好对 → 直接优化 → 模型

优势

流程更简单; 无需显式 RM/PPO; 训练通常更稳定。

局限

依赖离线偏好数据; 偏好目标变化时通常需要新的数据与再训练。

一句话记忆: RLHF “先训练评分员, 再强化学习”; DPO “不要评分员, 直接拿 Chosen > Rejected 训练模型”。

例题

RLHF 中的奖励模型 (RM) 是用来做什么的?

- A. 直接生成最终回答
- B. 为模型生成的回答打一个反映人类偏好程度的分数
- C. 用来替换预训练模型的注意力层
- D. 用来减少模型参数量

DPO 方法的创新之处在于:

- A. 引入了更复杂的强化学习算法
- B. 通过数学推导, 将偏好优化转化为直接的语言模型损失, 无需训练奖励模型
- C. 需要更多的显存来存储多个模型
- D. 只能用于图像生成任务

例题

在 RLHF 的 PPO 训练阶段，引入 KL 散度惩罚项的主要目的是：

- A. 加速模型收敛
- B. 增大模型的输出随机性
- C. 防止策略模型过度偏离参考模型，生成“投机取巧”的回答
- D. 减少训练数据的标注量

以下关于 DPO 和 RLHF 的说法，哪一项是错误的？

- A. DPO 训练比 RLHF 更稳定
- B. RLHF 的灵活性通常高于 DPO
- C. DPO 需要同时训练策略模型和奖励模型
- D. RLHF 的理论效果上限通常被认为高于 DPO

例题

关于大语言模型的训练流程，以下哪项分阶段描述最准确？

- A. 数据收集与预处理→大规模预训练→指令微调与对齐
- B. 在线推理→实时部署→用户体验评估
- C. 需求分析→产品设计→市场推广
- D. 只进行一次预训练即可



A

六大高频考点

认

AI与大模型基础

AI、机器学习、深度学习、大模型

它是什么

懂

模型原理

Token、Embedding、Transformer、注意力、预测

它怎么工作的

学

训练与对齐

预训练、微调、对齐、数据

它怎么学会的

用

提示与推理

部署、Prompt、RAG、思维链

怎么让它回答得更好

做

Agent与工具调用

Agent、API、工具、 workflow

怎么让它从说到做

守

安全、伦理与评价

偏见、隐私、版权、安全、评价

怎么判断它好不好、该不该这样用？

专题4：提示与推理

- 提示词工程（二轮高频） vs Prompt Tuning
- 上下文学习
- 思维链提示
- 检索增强生成RAG: Retrieval-Augmented Generation，它把大模型自身参数中的知识和外部知识库结合起来，在回答问题前先检索相关资料，再让模型基于这些资料生成答案

RAG检索增强

索引 (Indexing) → 检索 (Retrieval) → 生成 (Generation)

- **阶段1：索引 (Indexing)** —— 提前“备课”，建立知识库
- 这是离线阶段，在用户提问前完成，目的是把外部知识处理成模型易于检索的格式。
- 数据清洗与分块 (Chunking)：将PDF、Word、网页等原始文档，按照一定的逻辑（如段落、句子长度）切割成若干个小文本块 (Chunk)。太大则检索不精准，太小则上下文不完整。
- 向量化 (Embedding)：使用一个嵌入模型 (Embedding Model)，将每个文本块转换成一个高维的浮点数向量 (Vector)。这个向量代表了该文本块的“语义”。存储到向量数据库：将生成的向量和对应的原始文本，存入专门的向量数据库（如Pinecone、Milvus、Chroma）中，并建立索引以便快速检索。
- **阶段2：检索 (Retrieval)** —— 根据问题“翻书”找答案
- 这是在线阶段的第一步，在用户提问时执行。
- 问题向量化：用户输入一个问题 (Query)，系统使用同一个嵌入模型，将这个问题也转换成一个向量。
- 相似度搜索：在向量数据库中，通过计算向量间的余弦相似度或点积，找到与问题向量最接近的K个文本块（通常K=3~5）。这些文本块就是最可能包含答案的“候选证据”。
- **阶段3：生成 (Generation)** —— 参考“参考答案”进行作答
- 这是在线阶段的第二步，也是最关键的一步。
- 构建增强提示 (Augmented Prompt)：将检索到的相关文本块作为“上下文 (Context)”或“参考资料”，与用户的原始问题一起，组合成一个完整的Prompt。例如：
- “请根据以下参考信息回答用户问题。如果参考信息中没有答案，请直接说不知道，不要编造。”

例题

在RAG的标准工作流程中，索引（Indexing）阶段的主要目的是什么？

- A. 将用户问题转换成向量
- B. 将外部文档预处理成向量并存储，以便快速检索
- C. 训练一个专门的生成模型
- D. 对模型的输出进行排序

构建RAG的“增强提示（Augmented Prompt）”时，最关键的原则是：

- A. 将尽可能多的文档块塞入提示，增加信息量
- B. 清晰区分“参考信息”和“用户问题”，并指示模型在没有答案时不要编造
- C. 必须使用特定的提示模板，否则模型不执行检索
- D. 提示中需要包含模型的完整训练数据

例题

这是一个基于检索增强生成 (Retrieval-Augmented Generation, RAG) 的问答系统实现。该系统包含两个主要组件：

1. 检索器 (Retriever)：使用嵌入模型将文档转换为向量，根据查询语义检索最相关的文档
2. 生成器 (Generator)：基于检索到的相关文档和原始问题，生成准确的答案

请根据代码逻辑和注释：

self.corpus中所有文档
的向量表示

```
def build_index(self, corpus):  
    # 建立索引，所有知识库中的文档被转换为向量并建立索引  
    vecs_tensor = self._text_to_vector(corpus)  
    vecs_normalized = F.normalize(vecs_tensor, p=2, dim=1) # 归一化  
    return vecs_normalized.cpu() # 转到CPU存储  
  
def search(self, query_vec, topk=3):  
    if query_vec.dim() == 1:  
        query_vec = query_vec.unsqueeze(0)  
    query_normalized = F.normalize(query_vec, p=2, dim=1)  
    similarities = torch.matmul(query_normalized, self.embeddings.T) # 计算相似度  
    similarities = similarities.squeeze(0)  
  
    topk_scores, topk_indices = torch.topk(similarities, k=min(topk, len(self.corpus)))  
  
    results = [  
        # 通过idx从self.corpus中提取原文  
        {"text": self.corpus[idx], "score": float(topk_scores[i])}  
        for i, idx in enumerate(topk_indices)  
    ]  
    return results
```

```
Python  
import torch  
import torch.nn.functional as F  
from transformers import AutoTokenizer, AutoModel, AutoModelForCausalLM  
  
class Retriever: # 嵌入模型 分词器 语料库  
    def __init__(self, embedder, tokenizer, corpus):  
        self.embedder = embedder  
        self.tokenizer = tokenizer  
        self.corpus = corpus  
        self.embeddings = self.build_index(corpus)  
  
    def _text_to_vector(self, text_batch): # 向量  
        enc = [self.tokenizer.encode(text) for text in text_batch] # [21]  
        enc = {k: v.to(self.embedder.device) for k, v in enc.items()}  
  
        with torch.no_grad():  
            outputs = self.embedder(torch.cat(enc, dim=0)) # [22]  
            last_hidden = outputs.last_hidden_state  
            attn_mask = enc["attention_mask"]  
            sent_vecs = self._compute_sent_vecs(last_hidden, attn_mask) # [23]  
  
        return sent_vecs
```



```

def rag_answer(query, retriever, generator, gen_tokenizer, topk=3,
max_new_tokens=128):
    query_vec_tensor = retriever._text_to_vector([query])
    query_vec = query_vec_tensor.squeeze(0).cpu()

    docs = [24] 检索得到相应的文章

    context = "\n".join([d["text"] for d in docs]) if docs else ""
    prompt = f"Use the context to
answer.\nContext:\n{context}\n\nQ: {query}\nA:" 把检索结果增强到prompt中

    inputs = gen_tokenizer(prompt, return_tensors="pt")
    inputs = {k: v.to(generator.device) for k, v in
inputs.items()}
    pad_id = gen_tokenizer.pad_token_id if
gen_tokenizer.pad_token_id is not None else
gen_tokenizer.eos_token_id

    with torch.no_grad(): 推理阶段，不需要更新参数，不需要梯度
        output_ids = generator.generate( 根据Prompt进行自回归生成
            **inputs,
            do_sample=True, temperature=0.7, top_p=0.9,
            max_new_tokens=max_new_tokens,
            pad_token_id=pad_id,
            eos_token_id=gen_tokenizer.eos_token_id,
        )

    new_tokens = [25] output_ids 中把原来的输入Prompt部
    分切掉，只保留模型新生成的Token
    answer = gen_tokenizer.decode(new_tokens,
skip_special_tokens=True).strip()
    return answer

```

```

if __name__ == "__main__": 主函数的调用
    embedder_name = "../Qwen/Qwen3-Embedding-0.6B"
    retriever_tokenizer =
AutoTokenizer.from_pretrained(embedder_name)
    embedder = AutoModel.from_pretrained(embedder_name)
    generator_name = "../Qwen/Qwen3-0.6B"
    gen_tokenizer = AutoTokenizer.from_pretrained(generator_name)
    generator =
AutoModelForCausalLM.from_pretrained(generator_name)
    corpus = [
        "Paris is the capital of France.",
        "Tokyo is the capital of Japan.",
        "Beijing is the capital of China.",
    ]
    retriever = Retriever(embedder, retriever_tokenizer, corpus)
    query = "What is the capital of France?"
    ans = rag_answer(query, retriever, generator, gen_tokenizer,
topk=2)
    print("Q:", query)
    print("A:", ans)

```


例题

这是一个基于检索增强生成 (Retrieval-Augmented Generation, RAG) 的问答系统实现。该系统包含两个主要组件：

1. 检索器 (Retriever)：使用嵌入模型将文档转换为向量，根据查询语义检索最相关的文档
2. 生成器 (Generator)：基于检索到的相关文档和原始问题，生成准确的答案

请根据代码逻辑和注释提示，补全代码中的空位。

21. 在方法 `_text_to_vector` 中第一步是将输入的文本批次 `text_batch` 转换为模型可以理解的格式，以下代码正确的是：

- A. `self.embedder(text_batch, return_tensors="pt", truncation=True, max_length=512)`
- B. `self.tokenizer(text_batch, return_tensors="pt", truncation=True, max_length=512)`
- C. `self.tokenizer.encode(text_batch, return_tensors="pt", truncation=True, max_length=512)`
- D. `self.embedder(text_batch, return_tensors="pt", truncation=True, max_length=512)`

22. 此处需要进行前向计算获取隐藏向量，以下代码正确的是：

- A. `self.embedder(**enc)`
- B. `self.embedder.forward(enc["input_ids"])`
- C. `self.embedder(enc["input_ids"])`
- D. `self.embedder.forward(enc)`

```
Python
import torch
import torch.nn.functional as F
from transformers import AutoTokenizer, AutoModel,
AutoModelForCausalLM
```

```
class Retriever:           嵌入模型  分词器  语料库
    def __init__(self, embedder, tokenizer, corpus):
        self.embedder = embedder
        self.tokenizer = tokenizer
        self.corpus = corpus
        self.embeddings = self.build_index(corpus)

    def _text_to_vector(self, text_batch):
        enc = [21] 文本向量化
        enc = {k: v.to(self.embedder.device) for k, v in
enc.items()}           **enc, 将字典enc解包

        with torch.no_grad():
            outputs = [22] 前向计算获取隐藏向量
            last_hidden = outputs.last_hidden_state
            attn_mask = enc["attention_mask"]
            sent_vecs = [23] 池化得到句向量

        return sent_vecs
```

23. 此处需要池化得到句向量，以下代码正确的是：

- A. `last_hidden.mean(1)`
- B. `(last_hidden * attn_mask.unsqueeze(-1)).sum(1) / attn_mask.sum(1, keepdim=True)`
- C. `last_hidden[:, 0, :]`
- D. `(last_hidden * attn_mask).sum(1) / attn_mask.sum(1)`

```

def rag_answer(query, retriever, generator, gen_tokenizer, topk=3,
max_new_tokens=128):
    query_vec_tensor = retriever._text_to_vector([query])
    query_vec = query_vec_tensor.squeeze(0).cpu()

    docs = [24] 检索得到相应的文章

    context = "\n".join([d["text"] for d in docs]) if docs else ""
    prompt = f"Use the context to
answer.\nContext:\n{context}\n\nQ: {query}\nA:" 把检索结果增强到prompt中

    inputs = gen_tokenizer(prompt, return_tensors="pt")
    inputs = {k: v.to(generator.device) for k, v in
inputs.items()}
    pad_id = gen_tokenizer.pad_token_id if
gen_tokenizer.pad_token_id is not None else
gen_tokenizer.eos_token_id

    with torch.no_grad(): 推理阶段，不需要更新参数，不需要梯度
        output_ids = generator.generate( 根据Prompt进行自回归生成
            **inputs,
            do_sample=True, temperature=0.7, top_p=0.9,
            max_new_tokens=max_new_tokens,
            pad_token_id=pad_id,
            eos_token_id=gen_tokenizer.eos_token_id,
        )

    new_tokens = [25] output_ids 中把原来的输入Prompt部
    分切掉，只保留模型新生成的Token
    answer = gen_tokenizer.decode(new_tokens,
skip_special_tokens=True).strip()
    return answer

```

24. 此处需要进行检索得到对应文档，以下代码正确的是：

- A. retriever.index.search(query_vec, topk)
- B. retriever.search(query_vec, topk=topk)
- C. retriever.build_index(query_vec)
- D. retriever.generate(query_vec, topk=topk)

25. 此处需要对模型输出进行处理，以下代码正确的是：

- A. output_ids[0][-max_new_tokens:]
- B. output_ids[0][inputs["input_ids"].shape[1]:]
- C. output_ids[:, -max_new_tokens:]
- D. output_ids[:, inputs["input_ids"].shape[1]:]

专题4：推理

- 解码与推理策略
 - 贪心搜索 vs 束搜索：“贪心搜索”每一步只选概率最高的词，可能导致整体不够通顺；而“束搜索”保留多条候选路径，虽然计算更复杂，但更可能找到整体最优的句子。
 - 推理加速技术：vLLM、PagedAttention、FlashAttention等技术的核心作用。
- 复杂推理模式：模型如何解决多步骤问题，核心是思维链（Chain-of-Thought, CoT）及其相关技术
 - 长思维链：模型通过“一步一步思考”来分解复杂任务（如数学、逻辑、科学问题）的价值，例如增强可解释性和可靠性。
 - 高级搜索策略：Self-Consistency（自我一致性）和 Tree-of-Thoughts（思维树）等通过多路径搜索来提升推理准确性的方法。
- 推理模型的训练方法
 - 监督微调（SFT）：使用包含详细思维链的数据集对模型进行训练。
 - 基于强化学习（RL）的训练：包括对最终答案进行奖励的结果奖励模型，和对推理每一步进行奖励的过程奖励模型。其难点在于推理路径的搜索空间巨大，训练效率较低

专题4：推理

加速层面	核心技术	核心思想	解决的关键瓶颈	代表技术/案例
计算优化	FlashAttention	通过IO感知，避免反复读写显存（HBM），利用更快的片上缓存（SRAM）计算注意力。	显存带宽瓶颈，尤其在长文本场景下。	FlashAttention-2/3
	推测解码 (Speculative Decoding)	用一个“小模型”快速生成草稿，再用“大模型”并行验证，在保证质量的同时减少大模型的串行生成步数。	生成阶段的内存带宽瓶颈（受限于显存读取速度）。	vLLM中的实现、Diverse验证等
显存管理	PagedAttention	借鉴操作系统的分页机制，将KV Cache存储在非连续的物理页上，消除碎片。	显存碎片和显存浪费（过去需要预留连续空间）。	vLLM的核心创新
	KV Cache 优化	通过压缩、共享或更高效的位置编码，减少每个请求占用的KV Cache大小。	长上下文场景下的显存墙。	Multi-head Latent Attention (MLA)、KV Cache共享机制
模型轻量化	模型量化 (Quantization)	将模型权重从高精度（如FP16）转为低精度（如INT8、INT4），压缩模型体积，加快计算。	模型体积大导致的存储和计算压力。	GPTQ、AWQ、QLoRA
	模型剪枝 (Pruning)	移除模型中不重要的连接、神经元或注意力头，减小模型规模。	模型过度参数化带来的冗余计算。	结构化/非结构化剪枝
	知识蒸馏 (Knowledge Distillation)	用一个大模型（教师）指导一个小模型（学生）学习，让学生模型在更小的体积下复现大模型的能力。	直接从源头“制造”一个小而强的模型。	各种蒸馏技术
	高效架构	设计从根本就更省显存、计算更快的模型结构，例如用线性注意力替代标准注意力，或采用混合专家（MoE）架构。	传统Transformer架构的固有限制（如二次复杂度）。	

专题4：推理

TTS（Test-Time Scaling）测试时间拓展：在模型推理（测试）阶段，通过动态分配更多计算资源来撬动模型潜力，从而提升其在复杂任务上表现的技术总称。简单来说就是用时间/算力换质量，主要技术路径：

技术路径	核心思想	代表方法/技术
扩展推理路径	通过多次采样、让模型自我验证或修正答案，来提升正确率。这像让一位考生检查多遍考卷，或参考多个解法。	多数投票、Self-Consistency（自我一致性）、预算强制
并行搜索优化	更高效地管理多条并行的推理链（如思维树），确保算力能用在更有可能成功的路径上。	DORA等资源分配算法
动态资源分配	根据问题的不同部分或难易程度，灵活分配算力。对难题投入更多，对简单题则快速跳过，从而提升资源利用率。	SCALE框架、AgentTTS
推理效率优化	解决“长思维链”带来的显存和延迟瓶颈，让长时间思考在工程上变得可行。	AsyncSpade、FlashAttention等技术

例题

基于强化学习的推理能力优化方法面临的主要挑战是什么？

- A. 模型会因此完全丧失其原有的语言生成能力。
- B. 推理路径的搜索空间巨大，导致训练不稳定和效率低下。
- C. 强化学习会使模型的词汇量急剧减少。
- D. 该方法只能应用于数学推理，无法泛化到其他领域。

下列哪一项不属于测试时间拓展（Test Time Scaling, TTS）技术？

- A. 对单个测试样本进行多种变换（如翻转、裁剪），并将所有变换版本的预测结果进行集成。
- B. 在模型推理时，仍然随机丢弃部分神经元，进行多次前向传播并将结果平均，以模拟集成效果提升健壮性。
- C. 为每个测试样本定义一个辅助任务（如旋转图片预测），利用该样本对模型进行一次梯度下降微调，再完成主任务预测。
- D. 在一个大型源数据集上预训练模型，然后在一个较小的目标数据集上对模型的所有权重进行微调。

例题

关于思维链（Chain-of-Thought, CoT）提示方法的核心价值，以下描述最准确的是：

- A. 通过让模型一次性生成多个候选答案并投票，来提高最终答案的准确性。
- B. 通过引导模型在输出最终答案前，显式地生成中间推理步骤，从而提升复杂任务的推理准确性和可解释性。
- C. 通过在模型推理时随机丢弃部分神经元，来模拟集成学习的效果。
- D. 通过为每个测试样本定义一个辅助任务并进行梯度更新，来提升模型在该样本上的表现。

关于“贪心搜索”和“束搜索”这两种解码策略，以下哪种说法是最准确的？

- A. 贪心搜索因为每次都选最好的，所以总能生成最完美、最富有创造力的句子。
- B. 束搜索需要同时考虑多条路径，计算起来更复杂，但它更有可能找到一个整体上更通顺、更合理的句子。
- C. 束搜索的速度通常比贪心搜索更快，因为它是并行的。
- D. 这两种策略没有本质区别，无论用哪一种，AI生成的句子质量都完全一样。

六大高频考点

认

AI与大模型基础

AI、机器学习、深度学习、大模型

它是什么

懂

模型原理

Token、Embedding、Transformer、注意力、预测

它怎么工作的

学

训练与对齐

预训练、微调、对齐、数据

它怎么学会的

用

提示与推理

P部署、Prompt、RAG、思维链

怎么让它回答得更好

做

Agent与工具调用

Agent、API、工具、 workflow

怎么让它从说到做

守

安全、伦理与评价

偏见、隐私、版权、安全、评价

怎么判断它好不好、该不该这样用？

专题5: Agent-二轮高频考点

智能体 (Agent) 是一个以大语言模型为核心控制器的自主系统, 它能够:

- 感知环境 (接收用户指令、观察外部反馈)
- 规划任务 (将复杂目标分解为可执行的子步骤)
- 行动 (调用外部工具、API、数据库或操作物理/数字世界)
- 反思与迭代 (根据行动结果调整策略, 直到完成任务)

智能体是大语言模型 (LLM) 从“聊天机器人”进化为“数字执行者”的关键一步。如果说RAG是给模型配了参考书, 那么智能体就是给模型配上了手和脑, 让它不仅能说, 还能做

例题

在LLM智能体的架构中，以下哪个模块负责“将复杂目标拆解为可执行的子步骤”？

- A. 工具集 (Tools)
- B. 记忆模块 (Memory)
- C. 规划模块 (Planning)
- D. 核心控制器 (LLM)

关于Agent智能体的身份与角色设置，以下描述最准确的是：

- A. Agent的角色仅通过其名称进行区分，与系统提示词无关
- B. 为Agent设置明确的角色和职责边界，有助于防止其“越界”执行不属于其分工的任务
- C. Agent的角色在创建后即固定，无法在不同任务中复用或调整
- D. 所有Agent应共享相同的系统提示词，以确保行为一致性

六大高频考点

认

AI与大模型基础

AI、机器学习、深度学习、大模型

它是什么

懂

模型原理

Token、Embedding、Transformer、注意力、预测

它怎么工作的

学

训练与对齐

预训练、微调、对齐、数据

它怎么学会的

用

提示与推理

部署、Prompt、RAG、思维链

怎么让它回答得更好

做

Agent与工具调用

Agent、API、工具、工作流

怎么让它从说到做

守

安全、伦理与评价

偏见、隐私、版权、安全、评价

怎么判断它好不好、该不该这样用？

专题6：安全、伦理和评价

- 评测指标
- 困惑度：用来衡量语言模型对给定文本预测能力，核心逻辑是：模型对一段文本越熟悉，预测就越准确，困惑度就越低
- BLEU (Bilingual Evaluation Understudy)：双语评估，衡量生成文本与参考文本在n-gram级别上的重合度，核心是精确率，最早用于机器翻译
 - 把生成文本切成一个个1词、2词、3词、4词的片段 (n-gram)
 - 数一数这些片段中有多少也在参考译文中出现过
 - 比例越高 → BLEU越高 → 生成文本与参考译文在词面表达上越接近
- ROUGE (Recall-Oriented Understudy for Gisting Evaluation) 面向召回率的摘要评估，以召回率为核心的评估指标：参考文本中有多少信息被生成文本覆盖到了
- NDCG (Normalized Discounted Cumulative Gain) 衡量搜索引擎或推荐系统返回的排序列表质量，同时考虑相关性和位置：文档是否相关；排名越靠前的文档，权重越高。

例题

为了保障社会群体的“公平安全”，避免模型输出产生歧视性内容，最关键的缓解方法是在哪个阶段介入？

- A. 仅在模型推理部署后，通过实时过滤敏感词来屏蔽有害输出。
- B. 主要依靠扩大模型规模，期望其自动学习并消除偏见。
- C. 在模型预训练和微调阶段，系统性地清洗数据和引入公平性约束。
- D. 完全依赖第三方评测机构在发布前进行一次性的偏见评估。

关于语言模型的困惑度（Perplexity），以下描述正确的是：

- A. 困惑度越低，说明模型对测试数据的预测越不确定，性能越差。
- B. 困惑度是模型在训练集上的准确率直接转换而来的指标。
- C. 困惑度反映模型对一组文本的平均预测不确定性，值越低表示模型性能越好。
- D. 困惑度与模型的训练步数无关，只与模型架构有关。

备考策略



策略一：吃透大纲，精准锁定高频考点（抓大放小）

- 以下板块占分比高，建议优先吃透：
- 模型架构与核心机制：重点掌握Decoder-Only架构（GPT系列的自回归生成、因果掩码），以及Transformer中Query、Key、Value的投影关系。
- 训练与微调技术：理解LoRA（低秩适配，仅训练并联矩阵，显存占用低）、RLHF与DPO的区别，以及指令微调数据（指令+期望输出）的配对格式。
- 解码与评测指标：区分贪心搜索（每次选最优）和束搜索（保留多个候选）的优缺点；明确Pass@K（K个方案通过数）、困惑度（越低越好）的含义。

lmcc.ccf.org.cn

策略二：结合大纲构建知识树（系统化）

- Transformer、自注意力、涌现能力
- 指令微调、学习率预热、泛化与过拟合、批次大小影响
- HHH标准（Helpful, Honest, Harmless）、公平性、精确率与召回率
- 第一轮全是客观题，知识图谱越清晰，答题速度就越快

认

AI与大模型基础

懂

模型原理

Token、Embedding、Transformer、注意力、预测

学

训练与对齐

预训练、微调、对齐、数据

用

提示与推理

部署、Prompt、RAG、思维链

做

Agent与工具调用

Agent、API、工具、工作流

守

安全、伦理与评价

偏见、隐私、版权、安全、评价

- 策略三：用AI辅助学

大模型基础概念中的扩展法则，km扩展法则，chinchilla扩展法则，计算最优模型规模，数据规模与计算资源的权衡，具体是什么，详细讲解，并出5道选择题以及1道编程填空题

1. 扩展法则 (Scaling Laws) 的数学基础
2. KM 扩展法则：主张“模型优先”
3. Chinchilla 扩展法则：主张“数据与模型平衡”
4. 计算最优模型规模与资源权衡

关于扩展法则 (Scaling Laws) 的核心思想，以下哪项描述最准确？

- A. 模型性能与参数量呈线性关系。
- B. 模型的性能损失是模型大小、数据集大小和计算量这三个因素的可预测幂律函数。
- C. 只要无限增加训练数据，模型性能就能无限提升。
- D. 模型性能的提升完全取决于架构的创新。

建议——备考第二轮

- 1.1 考试流程总览
 - 整个考试流程包含三个核心环节，如下所示，请您知悉：
 - 开始考试 获取考试文件（通过指定命令） 作答（在VSCode中编辑文件） 提交答案（通过指定命令） 考试结束
- 1.2 考前准备：熟悉你的考试环境
 - 1. 桌面状态：座位的电脑已经开机，并打开了 Visual Studio Code (VSCode) 软件。
 - 2. 连接状态：VSCode 已经通过 SSH 远程连接到考试服务器。您会看到VSCode左下角显示为一个蓝色的连接状态栏（写着“SSH:[考生账号]”）。
 - 3. 你的工作：请确认考生账号（以LMCC-开头）是否正确，无需进行任何环境配置、连接或安装插件等操作。

首次LMCC青少年组交流活动承办单位：中国人民大学附属中学

2026-04-10 访问量：283 分享：  

由中国计算机学会（CCF）主办、中国人民大学附属中学承办的CCF青少年科学教育交流活动暨首次大模型能力认证（LMCC）青少年组交流活动定于2026年4月16日-20日在北京举办。本次活动旨在增进交流，培养对人工智能感兴趣的青少年；增进高校与优秀人工智能类、计算机类学生的紧密常规的交流，助力高校选拔优秀人才。

活动期间将在中国人民大学附属中学安排上机测试，后分别在中国人民大学、清华大学、北京大学三所高校举行讲座和参观丰富活动，还将走进阿里巴巴北京科技园C区阿里云展厅，开启一场系统性的人工智能前沿技术参观与互动之旅。